Quantum Computing, Quantum Machine Learning and Quantum Information Theories

Morten Hjorth-Jensen[1]

Department of Physics, University of Oslo, Norway[1]

April 2, 2025

Plans for the week of March 31-April 4, 2025

1. Discrete Fourier transforms (DFTs, reminder from last week)) and the fast Fourier Transform (FFT) (see https://en.wikipedia.org/wiki/Fast_Fourier_transform)
2. Quantum Fourier transforms (QFTs), reminder from last week
3. Setting up circuits for QFTs
4. Quantum phase estimation algorithm
5. Reading recommendation Hundt, Quantum Computing for Programmers, sections 6.1-6.4 on QFT.

Brief reminder on Fourier transforms

Last week we reviewed some basic properties of Fourier transforms and derived the expressions for the quantum Fourier variant. We restate some of these results here. For more information, see slides from previous week

Discrete Fourier transforms

The Discrete Fourier Transform is a tool used in many different fields and finds applications in for example signal processing. It has many many applications (in Speech Recognition, Data Processing, Optics, Acoustics). It is used in any application which performs Digital Signal Processing.
There is an efficient algorithm, called the Fast Fourier Transform (FFT), which uses the special relationship amongst the roots of unity to compute the entire DFT in $O(n \log n)$ time. Moreover, it is possible to take the DFT and recover the original polynomial in its coefficient form in $O(n \log n)$ time, by using the FFT to compute the inverse DFT.

Fast Fourier transform (FFT)

Because of the many important applications of DFT, the Fast Fourier Transform is ranked among the top ten algorithms in the 20th century. It came to light as a tool for Signal processing in 1965, when Cooley (of IBM) and Tukey (of Princeton) wrote a paper presenting the algorithm. However the basics of the algorithm were known long before that, for example, it was known to Gauss back in 1805. To read more about Fast Fourier transforms and similar topics, see for example Fast Fourier Transform - Algorithms and Applications. See also https://github.com/CompPhysics/QuantumComputingMachineLearning/blob/gh-pages/doc/Textbooks/fastfourier.pdf
For a discussion of FFT, see for example
https://en.wikipedia.org/wiki/Fast_Fourier_transform

The discrete Fourier transform (DFT)

The discrete Fourier transform takes as input a complex vector

$$\boldsymbol{x} = |\boldsymbol{x}\rangle = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_{n-2} \\ x_{n-1} \end{bmatrix}.$$

Each entry of the output vector $|\boldsymbol{y}\rangle$ is given by

$$y_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j e^{2\pi i j k / N}.$$

Output vector

The output is a complex vector

$$|\boldsymbol{y}\rangle = \frac{1}{\sqrt{n}} \begin{bmatrix} \sum_{j=0}^{n-1} x_j e^{2\pi i j 0/n} \\ \sum_{j=0}^{n-1} x_j e^{2\pi i j 1/n} \\ \sum_{j=0}^{n-1} x_j e^{2\pi i j 2/n} \\ \vdots \\ \sum_{j=0}^{n-1} x_j e^{2\pi i j (n-2)/n} \\ \sum_{j=0}^{n-1} x_j e^{2\pi i j (n-1)/n} \end{bmatrix}$$

Simple example

As an example we can have a set of complex numbers $\{x_0, \ldots, x_{n-1}\}$ with fixed length n, we can Fourier transform this as

$$y_k = \frac{1}{\sqrt{n}} \sum_{j=0}^{n-1} x_j \exp i(2\pi jk)/n,$$

leading to a new set of complex numbers $\{y_0, \ldots, y_{n-1}\}$.

Discrete Fourier Transformations

Consider two sets of complex numbers x_k and y_k with $k = 0, 1, \ldots, n-1$ entries. The discrete Fourier transform is defined as

$$y_k = \frac{1}{\sqrt{n}} \sum_{j=0}^{n-1} \exp\left(\frac{2\pi \imath jk}{n}\right) x_j.$$

As an example, assume $x_0 = 1$ and $x_1 = 1$. We can then use the above expression to find y_0 and y_1.

Simple example

With the above formula we get then

$$y_0 = \frac{1}{\sqrt{2}}\left(\exp\left(\frac{2\pi\imath 0 \times 1}{2}\right) \times 1 + \exp\left(\frac{2\pi\imath 0 \times 1}{2}\right) \times 2\right)$$
$$= \frac{1}{\sqrt{2}}(1+2) = \frac{3}{\sqrt{2}},$$

and

$$y_1 = \frac{1}{\sqrt{2}}\left(\exp\left(\frac{2\pi\imath 0 \times 1}{2}\right) \times 1 + \exp\left(\frac{2\pi\imath 1 \times 1}{2}\right) \times 2\right) =$$
$$= \frac{1}{\sqrt{2}}(1 + 2\exp(\pi\imath)) = -\frac{1}{\sqrt{2}}.$$

More details on Discrete Fourier transforms

Suppose that we have a vector f of n complex numbers, $f_k, k \in \{0, 1, \ldots, n-1\}$. Then the discrete Fourier transform (DFT) is a map from these n complex numbers to n complex numbers, the Fourier transformed coefficients $\tilde{f}_j$, given by

$$\tilde{f}_j = \frac{1}{\sqrt{n}} \sum_{k=0}^{n-1} \omega^{-jk} f_k$$

where $\omega = \exp\left(\frac{2\pi i}{N}\right)$.

Inverse DFT

The inverse DFT is given by

$$f_j = \frac{1}{\sqrt{n}} \sum_{k=0}^{n-1} \omega^{jk} \tilde{f}_k$$

To see this consider how the basis vectors transform. If $f_k^l = \delta_{k,l}$, then

$$\tilde{f}_j^l = \frac{1}{\sqrt{n}} \sum_{k=0}^{n-1} \omega^{-jk} \delta_{k,l} = \frac{1}{\sqrt{n}} \omega^{-jl}$$

Orthonormality

These DFT vectors are orthonormal, that is

$$\sum_{j=0}^{n-1} \tilde{f}^{l*}_j \tilde{f}^m_j = \frac{1}{n} \sum_{j=0}^{n-1} \omega^{jl} \omega^{-jm} = \frac{1}{N} \sum_{j=0}^{n-1} \omega^{j(l-m)}$$

This last sum can be evaluated as a geometric series, but beware of the $(l - m) = 0$ term, and yields

$$\sum_{j=0}^{n-1} \tilde{f}^{l*}_j \tilde{f}^m_j = \delta_{l,m}$$

Inverse transform

From this we can check that the inverse DFT does indeed perform the inverse transform:

$$f_j = \frac{1}{\sqrt{n}} \sum_{k=0}^{n-1} \omega^{jk} \tilde{f}_k = \frac{1}{\sqrt{n}} \sum_{k=0}^{n-1} \omega^{jk} \frac{1}{\sqrt{n}} \sum_{l=0}^{n-1} \omega^{-lk} f_l = \frac{1}{n} \sum_{k,l=0}^{n-1} \omega^{(j-l)k} f_l = \sum_{l=0}^{n-1} \delta$$

QFT, short review from last week

1. QFTs are the quantum analogue of the Discrete Fourier Transform (DFT).
2. They play a crucial role in quantum algorithms like Shor's Algorithm and the Quantum Phase Estimation (QPE) algorithm
3. QFTs provide an *exponential speedup* over classical Fourier Transform methods.

Advantage 1: Exponential Speedup

1. Classical Discrete Fourier Transforms (DFTs) require $O(N^2)$ operations.
2. The Fast Fourier Transforms (FFTs) improve this to $O(N \log N)$ operations.
3. QFTs reduce complexity to $O((\log N)^2)$ using quantum gates.

Advantage 2: Quantum Parallelism

QFTs act on a *superposition* of states, processing all inputs simultaneously. This is crucial in algorithms like:

1. Shor's Algorithm for factoring large numbers.
2. Quantum Phase Estimation (QPE) for eigenvalue extraction.

Advantage 3: Compact Circuit Implementation

1. QFTs require only *Hadamard gates and controlled-phase gates*.
2. QFTs are highly efficient for *quantum hardware*.

Advantage 4: Applications in Quantum Computing

1. **Shor's Algorithm:** Uses QFT to find periodicity in modular exponentiation.
2. **Quantum Phase Estimation (QPE):** Extracts eigenvalues of unitary matrices.
3. **Quantum Signal Processing:** Enables spectral analysis on quantum data.

Advantage 5: Reduced Memory Complexity

1. Classical DFTs require $O(N)$ space.
2. QFTs store Fourier-transformed coefficients *implicitly* in qubit amplitudes.
3. Only $O(n)$ qubits are needed for a size $N = 2^n$ transformation.

Advantage 6: Quantum Signal Processing

QFTs enable applications such as:

1. Quantum spectral analysis.
2. Quantum image processing.
3. Quantum filtering and denoising.

These can enhance AI, cryptography, and data processing.

From DFT to QFT

The Quantum Fourier Transform (QFT) has mathematically the same equation as starting point but the notation is generally different. We generally compute the quantum Fourier transform on a set of orthonormal basis state vectors $|0\rangle, |1\rangle, ..., |N-1\rangle$. The linear operator defining the transform is given by the action on basis states

$$|j\rangle \mapsto \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi ijk/N} |k\rangle.$$

In terms of arbitrary states

This can be written on arbitrary states,

$$\sum_{j=0}^{N-1} x_j |j\rangle \mapsto \sum_{k=0}^{N-1} y_k |k\rangle$$

where each amplitude y_k is the discrete Fourier transform of x_j.

Unitarity

The quantum Fourier transform is unitary. Taking $N = 2^n$, for n qubits gives us the orthonormal (computational) basis

$$|0\rangle, |1\rangle, ..., |2^n - 1\rangle.$$

Full QFT

So what is the full quantum Fourier transform? It is the transform which takes the amplitudes of a N dimensional state and computes the Fourier transform on these amplitudes (which are then the new amplitudes in the computational basis.) In other words, the QFT enacts the transform

$$\sum_{x=0}^{N-1} a_x|x\rangle \rightarrow \sum_{x=0}^{N-1} \tilde{a}_x|x\rangle = \sum_{x=0}^{N-1} \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} \omega_N^{-xy} a_y|x\rangle$$

Explicit transform

It is easy to see that this implies that the QFT performs the following transform on basis states:

$$|x\rangle \rightarrow \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} \omega_N^{-xy} |y\rangle$$

Thus the QFT is given by the matrix

$$U_{QFT} = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} \omega_N^{-yx} |y\rangle\langle x|$$

Unitarity

The last matrix is unitary. Let's check this:

$$
\begin{aligned}
U_{QFT}\,U_{QFT}^{\dagger} &= \frac{1}{N}\sum_{x=0}^{N-1}\sum_{y=0}^{N-1}\omega_N^{yx}|x\rangle\left\langle y\right|\sum_{x'=0}^{N-1}\sum_{y'=0}^{N-1}\omega_N^{-y'x'}\left|y'\right\rangle\langle x'| \\
&= \frac{1}{N}\sum_{x,y,x',y'=0}^{N-1}\omega_N^{yx-y'x'}\delta_{y,y'}|x\rangle\left\langle x'\right| = \frac{1}{N}\sum_{x,y,x'=0}^{N-1}\omega_N^{y(x-x')}\left|x\right\rangle \\
&= \sum_{x,x'=0}^{N-1}\delta_{x,x'}|x\rangle\langle x'| = I
\end{aligned}
$$

Importance of QFT

The QFT is a very important transform in quantum computing. It can be used for all sorts of cool tasks, including, as we shall see in Shor's algorithm. But before we can use it for quantum computing tasks, we should try to see if we can efficiently implement the QFT with a quantum circuit. Indeed we can and the reason we can is intimately related to the fast Fourier transform.

Circuit QFT

Let's derive a circuit for the QFT when $N = 2^n$. The QFT performs the transform

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} \omega_N^{-xy} |y\rangle$$

Then we can expand out this sum

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \sum_{y_1,y_2,\ldots,y_n \in \{0,1\}} \omega_N^{-x \sum_{k=1}^{n} 2^{n-k} y_k} |y_1, y_2, \ldots, y_n\rangle$$

Expanding the exponential

Expanding the exponential of a sum to a product of exponentials and collecting these terms in from the appropriate terms we can express this as

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \sum_{y_1,y_2,\ldots,y_n \in \{0,1\}} \bigotimes_{k=1}^{n} \omega_N^{-x2^{n-k}y_k} |y_k\rangle$$

Rearranging

We can rearrange the sum and products as

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \bigotimes_{k=1}^{n} \left[\sum_{y_k \in \{0,1\}} \omega_N^{-x 2^{n-k} y_k} |y_k\rangle \right]$$

Expanding this sum yields

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \bigotimes_{k=1}^{n} \left[|0\rangle + \omega_N^{-x 2^{n-k}} |1\rangle \right]$$

Notation for binary fraction

But now notice that $\omega_N^{-x2^{n-k}}$ is not dependent on the higer order bits of x. It is convenient to adopt the following expression for a binary fraction:

$$0.x_l x_{l+1} \ldots x_n = \frac{x_l}{2} + \frac{x_{l+1}}{4} + \cdots + \frac{x_n}{2^{n-l+1}}$$

More notations

Then we can see that

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \left[|0\rangle + e^{-2\pi i 0.x_n}|1\rangle\right] \otimes \left[|0\rangle + e^{-2\pi i 0.x_{n-1}x_n}|1\rangle\right] \otimes \cdots \otimes \left[|0\rangle + e^{-2\pi}\right.$$

This is a very useful form of the QFT for $N = 2^n$. Why? Because we see that only the last qubit depends on the the values of all the other input qubits and each further bit depends less and less on the input qubits. Further we note that $e^{-2\pi i 0.a}$ is either +1 or -1 , which reminds us of the Hadamard transform.

Deriving a circuit

So how do we use this to derive a circuit for the QFT over $N = 2^n$? Take the first qubit of $|x_1, \ldots, x_n\rangle$ and apply a Hadamard transform. This produces the transform

$$|x\rangle \to \frac{1}{\sqrt{2}} \left[|0\rangle + e^{-2\pi i 0 \cdot x_1}|1\rangle\right] \otimes |x_2, x_3, \ldots, x_n\rangle$$

Rotation gate

Now define the rotation gate

$$R_k = \begin{bmatrix} 1 & 0 \\ 0 & \exp\left(\frac{-2\pi i}{2^k}\right) \end{bmatrix} \tag{30}$$

If we now apply controlled R_2, R_3, etc. gates controlled on the appropriate bits this enacts the transform

$$|x\rangle \rightarrow \frac{1}{\sqrt{2}}\left[|0\rangle + e^{-2\pi i 0.x_1 x_2 \ldots x_n}|1\rangle\right] \otimes |x_2, x_3, \ldots, x_n\rangle$$

Proceeding

Thus we have reproduced the last term in the QFTed state. Of course now we can proceed to the second qubit, perform a Hadamard, and the appropriate controlled R_k gates and get the second to last qubit. Thus when we are finished we will have the transform

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \left[|0\rangle + e^{-2\pi i 0.x_1 x_2 \cdots x_n}|1\rangle\right] \otimes \left[|0\rangle + e^{-2\pi i 0.x_1 x_2 \cdots x_{n-1}}|1\rangle\right] \otimes \cdots \otimes \left[|0\right.$$

Reversing the order of these qubits will then produce the QFT!

Two-qubit system

These notes will be extended, see Hundt section 6.2

Two-qubit case

For a two-qubit system $n = 2$, the QFT transformation is:

$$\mathrm{QFT}|x_0 x_1\rangle = \frac{1}{2}\sum_{y=0}^{3} e^{2\pi i x y/4}|y\rangle,$$

where $x = x_0 x_1$ (binary representation).

QFT matrix

The QFT on two qubits has the matrix:

$$U_{\mathrm{QFT}} = \frac{1}{2}\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -1 \\ 1 & -i & -1 & i \end{bmatrix}$$

Exercise: apply this to the $|01\rangle$ (binary 1) and $|10\rangle$ (binary 2) stats.
Hint: see Hundt section 6.2 and circuit figures as well.

Three-qubit case

See whiteboard notes April 2.

Four-qubit system

The basis states are

$$|j_1 j_2 j_3 j_4\rangle$$

where j_k is either 0 or 1. We have

$$\begin{aligned}
0.j_3 &= \frac{j_3}{2} \\
0.j_2 j_3 &= \frac{j_2}{2} + \frac{j_3}{4} \\
0.j_1 j_2 j_3 &= \frac{j_1}{2} + \frac{j_2}{4} + \frac{j_3}{8} \\
0.j_0 j_1 j_2 j_3 &= \frac{j_0}{2} + \frac{j_1}{4} + \frac{j_2}{8} + \frac{j_3}{16}
\end{aligned}$$

QFT acts as follows

The quantum Fourier transform acts as follows:

$$|j_1 j_2 j_3 j_4\rangle \mapsto \frac{1}{\sqrt{2^{4/2}}} \left(|0\rangle + e^{2\pi i 0.j4}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i 0.j_3 j4}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i 0}\right.$$

Quantum circuits

To compose a quantum circuit that calculates the quantum Fourier transform we use the operators

$$R_k = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{bmatrix}.$$

Rotation gates

In this example, the R_k gates are:

$$R_1 = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^0} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad R_2 = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^2} \end{bmatrix}, \quad R_3 = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^3} \end{bmatrix}$$

Using the Hadamard gate

The Hadamard gate on a single qubit creates an equal superposition of its basis states, assuming it is not already in a superposition, such that

$$H|0\rangle = \frac{1}{\sqrt{2}}\left(|0\rangle + |1\rangle\right), \quad H|1\rangle = \frac{1}{\sqrt{2}}\left(|0\rangle - |1\rangle\right)$$

The R_k gate simply adds a phase if the qubit it acts on is in the state $|1\rangle$

$$R_k|0\rangle = |0\rangle, \quad R_k|1\rangle = e^{2\pi i/2^k}|1\rangle$$

Since all this gates are unitary, the quantum Fourier transfrom is also unitary.

Algorithm

Assume we have a quantum register of n qubits in the state $|j_1 j_2 \dots j_n\rangle$. Applying the Hadamard gate to the first qubit produces the state

$$H|j_1 j_2 \dots j_n\rangle = \frac{\left(|0\rangle + e^{2\pi i 0.j_1}|1\rangle\right)}{2^{1/2}}|j_2 \dots j_n\rangle.$$

Binary fraction

Here we have made use of the binary fraction to represent the action of the Hadamard gate

$$\exp 2\pi i 0.j_1 = -1,$$

if $j_1 = 1$ and $+1$ if $j_1 = 0$.

Controlled rotation gate

Furthermore we can apply the controlled-R_k gate, with all the other qubits j_k for $k > 1$ as control qubits to produce the state

$$\frac{\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle\right)}{2^{1/2}}|j_2 \ldots j_n\rangle$$

Next we do the same procedure on qubit 2 producing the state

$$\frac{\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.j_2 \ldots j_n}|1\rangle\right)}{2^{2/2}}|j_2 \ldots j_n\rangle$$

Applying to all qubits

Doing this for all n qubits yields state

$$\frac{\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.j_2 \ldots j_n}|1\rangle\right)\ldots\left(|0\rangle + e^{2\pi i 0.j_n}|1\rangle\right)}{2^{n/2}}|j_2 \ldots j_n\rangle$$

At the end we use swap gates to reverse the order of the qubits

$$\frac{\left(|0\rangle + e^{2\pi i 0.j_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.j_{n-1} j_n}|1\rangle\right)\ldots\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle\right)}{2^{n/2}}|j_2 \ldots j_n\rangle$$

This is just the product representation from earlier, obviously our desired output.

QFT example codes

This code sets up the QFT for n qubits.

```
import numpy as np
def qft(n):

    # Inputs n qubits and returns QFT matrix of size (2^n, 2^n)
    dim = 2 ** n  # Dimension of the Hilbert space
    omega = np.exp(2j * np.pi / dim)  # Primitive root of unity
    # Initialize QFT matrix
    QFT = np.zeros((dim, dim), dtype=complex)
    # Construct the QFT matrix
    for i in range(dim):
        for j in range(dim):
            QFT[i, j] = omega ** (i * j) / np.sqrt(dim)
    return QFT

def apply_qft(state):
    n = int(np.log2(len(state)))  # Number of qubits
    qft_matrix = qft(n)
    return qft_matrix @ state
# Example: 3-qubit system
n_qubits = 3  # QFT on 3 qubits (8 dimensions)
dim = 2 ** n_qubits

# Define an example quantum state (|5⟩ in computational basis)
state = np.zeros(dim, dtype=complex)
state[5] = 1  # |5⟩ = [0,0,0,0,0,1,0,0]

print("Initial State |5⟩:", state)
# Apply QFT
```

Example using Qiskit

```
import numpy as np
from qiskit import QuantumCircuit, Aer, execute
def qft(circuit, n):
    # Apply the Quantum Fourier Transform to the first n qubits in the
    # Apply Hadamard gates and controlled rotations
    for j in range(n):
        circuit.h(j)
        for k in range(j + 1, n):
            circuit.cp(np.pi / 2**(k - j), k, j)
    # Swap the qubits to reverse their order
    for i in range(n // 2):
        circuit.swap(i, n - i - 1)
# Number of qubits
n = 4
# Create a quantum circuit with n qubits
qc = QuantumCircuit(n)
# Apply QFT to the quantum circuit
qft(qc, n)
# Draw the resulting circuit
print("Quantum Circuit for QFT:")
print(qc.draw(output='text'))
# Run the quantum circuit on a statevector simulator backend
# Execute the quantum circuit and get results
result = job.result()
output_state_vector = result.get_statevector()
print("\nOutput State Vector:")
print(output_state_vector)
```

Example using Pennylane

```
import pennylane as qml
from pennylane import numpy as np
# Define the number of qubits
n_qubits = 3
# Create a device using default.qubit
dev = qml.device("default.qubit", wires=n_qubits)
# Define the Quantum Fourier Transform as a Quantum Function
@qml.qnode(dev)
def qft_circuit():
    for j in range(n_qubits):
        # Apply the Hadamard gate to the j-th qubit
        qml.Hadamard(wires=j)
        # Apply controlled phase shifts
        for k in range(j + 1, n_qubits):
            qml.ControlledPhaseShift(np.pi / 2 ** (k - j), wires=[k, j]

    # PennyLane natively returns the state, but if you want measurement
    return qml.probs(wires=range(n_qubits))

# Execute the QFT circuit
probabilities = qft_circuit()
# Print the resulting state probabilities
print("Probabilities after performing QFT:")
print(probabilities)
```

Introduction to Quantum Phase Estimation (QPE)

The QPE is a fundamental quantum algorithm used to estimate the phase ϕ in eigenvalue equations of unitary operators. Given a unitary operator U and an eigenvector $|\psi\rangle$ such that:

$$U|\psi\rangle = \exp 2\pi i\phi|\psi\rangle$$

the goal of QPE is to estimate ϕ.
The QPE provides an estimate of ϕ with high probability. It is a crucial subroutine for algorithms like

1. Shor's factoring algorithm and
2. quantum many-body simulations.

Mathematical Background

The goal of QPE is to estimate the phase $\phi \in [0, 1)$ where:

$$U|\psi\rangle = \exp 2\pi i\phi|\psi\rangle.$$

The Quantum Fourier Transform on n qubits is defined as (see above):

$$|x\rangle \to \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} e^{2\pi ixy/2^n}|y\rangle.$$

The QFT is essential for extracting phase information in QPE.

Quantum Circuit and Working

The quantum circuit for QPE consists of two quantum registers:

1. The first register with n qubits is initialized to $|0\rangle^{\otimes n}$.
2. second register is initialized to the eigenstate $|\psi\rangle$ of U.

Figure to be added

Derivation of the Algorithm, Step 1: Initialization

The system starts in the state:

$$|0\rangle^{\otimes n} \otimes |\psi\rangle.$$

Applying Hadamard gates to the first register creates a superposition:

$$\frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} |k\rangle \otimes |\psi\rangle.$$

Step 2: Controlled Unitary Operations

Controlled-U^{2^j} gates apply the operation U with exponential powers:

$$\frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} |k\rangle \otimes U^k |\psi\rangle.$$

For eigenstate $|\psi\rangle$, this becomes:

$$\frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} e^{2\pi i \phi k} |k\rangle \otimes |\psi\rangle.$$

Step 3: Quantum Fourier Transform (QFT)

Applying QFT to the first register transforms the state to:

$$\sum_{k=0}^{2^n-1} c_k |k\rangle,$$

where coefficients c_k are peaked near $k \approx 2^n \phi$.

Step 4: Measurement

Measuring the first register gives an n-bit estimate of ϕ with high probability.

Simple code which implements the QPE

```
import numpy as np
def hadamard(n):
    # Creates an n-qubit Hadamard gate as a matrix.
    H = np.array([[1, 1], [1, -1]]) / np.sqrt(2)
    H_n = H
    for _ in range(n - 1):
        H_n = np.kron(H_n, H)  # Tensor product to expand Hadamard gate
    return H_n

def qft(n):
    # Creates an n-qubit Quantum Fourier Transform (QFT) matrix.
    N = 2**n
    omega = np.exp(2j * np.pi / N)
    qft_matrix = np.array([[omega**(i * j) for j in range(N)] for i in r
    return qft_matrix

def inverse_qft(n):
    # Creates an n-qubit inverse Quantum Fourier Transform (QFT†) matrix
    return np.conj(qft(n)).T  # Hermitian transpose of QFT

def controlled_unitary(U, control, target, n):
    # Creates an n-qubit controlled unitary matrix
    I = np.eye(2**n)  # Identity matrix
    CU = np.copy(I)
    for i in range(2**n):
        if (i >> control) & 1:  # Check if control qubit is |1⟩
            CU[i, :] = np.kron(np.eye(2**target), U).dot(I[i, :])
    return CU
```

Code which implements the QPE using Qiskit

```
from qiskit import QuantumCircuit, Aer, transpile, assemble, execute
from qiskit.visualization import plot_histogram
import numpy as np

def qpe(unitary, num_counting_qubits):
    """
    Quantum Phase Estimation Algorithm.
    Args:
        unitary (QuantumCircuit): The unitary operator whose phase we es
        num_counting_qubits (int): Number of qubits in the counting regi

    Returns:
        QuantumCircuit: QPE quantum circuit
    """
    n = num_counting_qubits
    qc = QuantumCircuit(n + 1, n)  # n counting qubits + 1 eigenstate qu
    # Step 1: Apply Hadamard to counting qubits
    for qubit in range(n):
        qc.h(qubit)
    # Step 2: Apply controlled-U^2^j operations
    for j in range(n):
        power = 2**j
        controlled_U = unitary.control(1).power(power)
        qc.append(controlled_U, [j] + [n])  # Control: j, Target: n
    # Step 3: Apply inverse QFT
    qc.append(qft_dagger(n), range(n))
    # Step 4: Measure counting qubits
    qc.measure(range(n), range(n))
    return qc
```

Code which implements the QPE using PennyLane

```
import pennylane as qml
import numpy as np
def qft(n):
    # Applies the inverse Quantum Fourier Transform (QFT†) circuit
    for i in range(n):
        qml.Hadamard(wires=i)
        for j in range(i):
            qml.CPhase(-np.pi / (2 ** (i - j)), wires=[j, i])
    for i in range(n // 2):
        qml.SWAP(wires=[i, n - i - 1])

def controlled_unitary(U, control, target):
    # Applies a controlled-unitary operation
    qml.ctrl(U, control=control)(wires=target)


def qpe(phi, num_counting_qubits):
    # Quantum Phase Estimation circuit in PennyLane.
    total_qubits = num_counting_qubits + 1
    dev = qml.device("default.qubit", wires=total_qubits, shots=1000)
    @qml.qnode(dev)
    def circuit():
        # Initialize the counting register in |0> and eigenstate registe
        qml.PauliX(wires=num_counting_qubits)  # Set last qubit to |1>
        # Apply Hadamard to counting qubits
        for qubit in range(num_counting_qubits):
            qml.Hadamard(wires=qubit)
        # Apply controlled-U^2^j operations
        for j in range(num_counting_qubits):
```

Applications of QPE

The QPE is a foundational algorithm with several key applications:

1. **Factoring and Cryptography:** Integral to Shor's algorithm for integer factoring.
2. **Quantum Simulations:** Estimating eigenvalues of Hamiltonians in many-body physics.
3. **Amplitude Estimation:** Enhances algorithms like Grover's search.

Challenges and Practical Considerations

1. **Qubit Requirements:** Requires a large number of qubits for high precision.
2. **Gate Depth:** Controlled-U^{2^j} operations increase gate complexity.
3. **Noise Sensitivity:** Errors in QFT or controlled gates impact accuracy.

It is a powerful quantum algorithm for extracting phase information of unitary operators. It forms the foundation for many advanced quantum algorithms and demonstrates the power of quantum Fourier transforms in computational tasks. For many-body simulations it is however not as efficient as the VQE algorithm.